

PROGRAMAÇÃO EM PYTHON



PROGRAMAÇÃO EM PYTHON



Por meio de uma parceria com o SENAI-SP, a Prefeitura de Santana de Parnaíba está disponibilizando um curso focado em Python, com o objetivo de preparar os alunos para o desenvolvimento de programas e aplicações.



PROGRAMAÇÃO EM PYTHON



Plano de Aula

- Conteúdo:**
- Automação e Manipulação de Arquivos;
 - Automação de Tarefas com Python;
 - Leitura e gravação de arquivos:
 - txt, CSV, JSON;
 - Atividades;

Início:

As informações deste conteúdo visam compreender o conteúdo do curso.



PROGRAMAÇÃO EM PYTHON



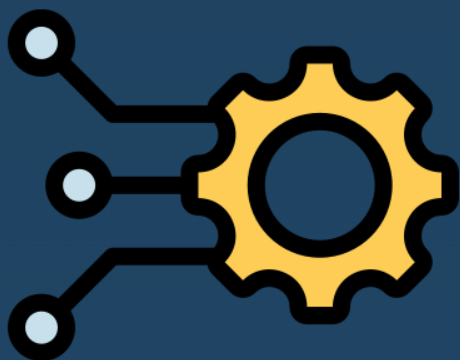
Objetivo

Capacitar profissionais para desenvolver aplicações em linguagem Python, por meio de técnicas de programação, seguindo boas práticas, procedimentos e normas.

PROGRAMAÇÃO EM PYTHON



Automação e Manipulação de Arquivos



Definição

Automação é o uso da **tecnologia para executar tarefas de forma automática**, sem a necessidade de intervenção humana constante. Com o auxílio de linguagens como o **Python**, é possível criar programas que realizam atividades repetitivas de maneira rápida, eficiente e com menos erros.

Manipulação de arquivos é o processo de criar, ler, modificar e organizar arquivos por meio de **programação**. Essa prática permite **automatizar tarefas como leitura de dados, geração de relatórios e organização de informações**, sendo muito útil no dia a dia.

PROGRAMAÇÃO EM PYTHON



Automação e Manipulação de Arquivos



Automação de Tarefas com Python

A **automação de tarefas** consiste em utilizar a **programação para executar atividades de forma automática**, reduzindo a necessidade de intervenção humana.

Com o uso do **Python**, é possível transformar tarefas repetitivas em processos rápidos e eficientes.

Principais vantagens:

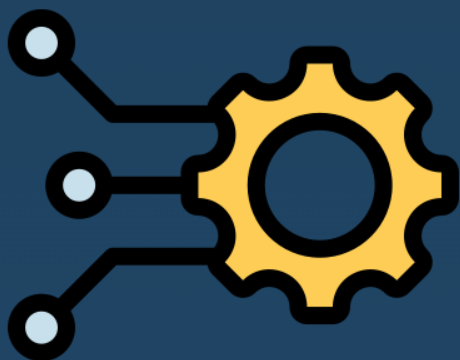
- 🕒 Economia de tempo
- 🎯 Redução de erros
- 🔁 Execução de tarefas repetitivas automaticamente
- 🚀 Aumento de produtividade

💡 Exemplo: renomear vários arquivos ou organizar dados automaticamente.

PROGRAMAÇÃO EM PYTHON



Automação e Manipulação de Arquivos



Aplicações da Automação

A **automação** está presente em diversas áreas do nosso dia a dia, desde atividades simples até processos mais complexos em empresas.

Onde podemos aplicar:

- 📁 Organização de arquivos: **mover**, **renomear** e **limpar pastas**
- 📊 Estudos: gerar relatórios, analisar notas
- 🌐 Internet: coleta de dados e envio automático de mensagens
- 🏢 Empresas: controle de estoque, processamento de dados

👉 Com Python, tarefas que levariam horas podem ser feitas em segundos.

PROGRAMAÇÃO EM PYTHON



Função `.open()` no Python



A função `open()` é utilizada para abrir arquivos no **Python**, permitindo realizar operações de **leitura**, **escrita** e **edição** de textos.

Estrutura básica:

```
1 open("arquivo.txt", "modo")
```

- **"arquivo.txt"** → nome do arquivo
- **"modo"** → define como o arquivo será utilizado

PROGRAMAÇÃO EM PYTHON



Função `.open()` no Python



Modos de Leitura

Modo	Descrição
"r"	Leitura do arquivo
"rt"	Leitura de texto (padrão)



Exemplo:

```
1 arquivo = open("texto.txt", "r")
2 conteudo = arquivo.read()
3 print(conteudo)
4 arquivo.close()
```

PROGRAMAÇÃO EM PYTHON



Função .open() no Python



Modos de Escrita

Modo	Descrição
"w"	Escreve no arquivo (apaga conteúdo anterior)
"a"	Adiciona conteúdo sem apagar
"x"	Cria um novo arquivo
"wt"	Escrita em texto



Exemplo:

```
1 arquivo = open("texto.txt", "a")
2 arquivo.write("Nova linha adicionada")
3 arquivo.close()
```

PROGRAMAÇÃO EM PYTHON



Função `.open()` no Python



Outros Modos Úteis

Modo	Descrição
"r+"	Leitura e escrita
"w+"	Escrita e leitura
"a+"	Adiciona e lê conteúdo



Importante:

Após utilizar um **arquivo**, é recomendado usar `close()` ou a estrutura `with open()` para fechar o arquivo automaticamente.

PROGRAMAÇÃO EM PYTHON



Automação de Texto com Arquivos .txt



Automação de texto

A **automação** de texto permite trabalhar com arquivos de forma rápida e automática, como **leitura**, **escrita** e edição de conteúdos.

Arquivos .txt são os mais simples, pois armazenam apenas texto puro.

Aplicações:

- Leitura de conteúdos automaticamente
- Criação de arquivos de texto
- Análise simples (contar palavras, buscar termos)

PROGRAMAÇÃO EM PYTHON



Automação de Texto com Arquivos .txt

Vamos
realizar
juntos!



 01-Texto.py X

material >  01-Texto.py > ...

```
1  # Ler um arquivo de texto
2  with open("texto.txt", "r") as arquivo:
3      conteudo = arquivo.read()
4      print(conteudo)
5
6  # Escrever em um arquivo
7  with open("texto.txt", "w") as arquivo:
8      arquivo.write("Olá, isso foi gerado automaticamente!")
9
```

Esse código demonstra como o **Python** interage com o **sistema de arquivos**, permitindo que o programa primeiro **leia** informações armazenadas e, em seguida, **sobrescreva** o conteúdo com uma nova mensagem.

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .CSV



Automação com Arquivos .CSV

Arquivos .CSV (**C**omma **S**eparated **V**alues) são usados para armazenar **dados** em formato de tabela (**linhas** e **colunas**), muito comuns em **planilhas**.

Aplicações do CSV

-  Planilhas e relatórios
-  Controle de notas escolares
-  Cadastro de produtos
-  Análise de dados

💡 O formato CSV é um dos mais utilizados para troca de dados entre sistemas e planilhas.

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .CSV



Manipulação de Arquivos CSV no Python

A biblioteca `csv` do Python permite trabalhar com arquivos `.csv` (**C**omma **S**eparated **V**alues), muito utilizados para armazenar **dados** em formato de tabela.



Importando a biblioteca



```
1 import csv
```

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .CSV



Leitura de Arquivos CSV

A **leitura** é feita utilizando `csv.reader()`, permitindo acessar **linha por linha do arquivo**.

Função	Descrição
<code>csv.reader()</code>	Lê o conteúdo do CSV
<code>next()</code>	Pula linhas específicas
<code>for linha in leitor</code>	Percorre os dados

Exemplo:

```
1 import csv
2
3 with open("dados.csv", "r") as arquivo:
4     leitor = csv.reader(arquivo)
5
6     for linha in leitor:
7         print(linha)
```

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .CSV



Escrita de Arquivos CSV

A **escrita** é feita utilizando `csv.writer()`, permitindo **adicionar linhas e dados automaticamente**.

Função	Descrição
<code>csv.writer()</code>	Cria um objeto de escrita
<code>writerow()</code>	Escreve uma linha
<code>writerows()</code>	Escreve várias linhas



Exemplo:

```
1 import csv
2
3 with open("dados.csv", "w", newline="") as arquivo:
4     escritor = csv.writer(arquivo)
5
6     escritor.writerow(["Nome", "Nota"])
7     escritor.writerow(["Ana", 9])
```

PROGRAMAÇÃO EM PYTHON



Automação de Texto com Arquivos .txt

Vamos
realizar
juntos!



02-Csv.py X

material > 02-Csv.py > ...

```
1 import csv
2
3 # Ler um arquivo CSV
4 with open("dados.csv", "r") as arquivo:
5     leitor = csv.reader(arquivo)
6     for linha in leitor:
7         print(linha)
8
9 # Escrever em um CSV
10 with open("dados.csv", "w", newline="") as arquivo:
11     escritor = csv.writer(arquivo)
12     escritor.writerow(["Nome", "Nota"])
13     escritor.writerow(["Ana", 9])
14
```

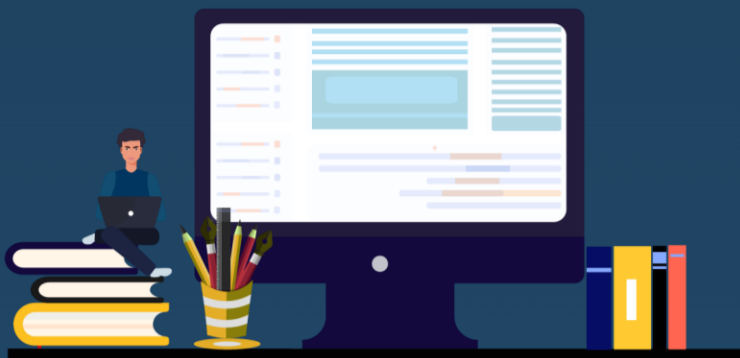
Este código utiliza a biblioteca `csv` para **processar dados estruturados**, permitindo percorrer as linhas de uma planilha existente e criar um novo arquivo organizado em **colunas e registros**.

PROGRAMAÇÃO EM PYTHON



Apresentação da situação de aprendizagem

1



Sistema de Notas - **FABPROG**

Imagine que você foi contratado pela escola **FABPROG** e precisa desenvolver um sistema simples para organizar as informações dos alunos de forma automática.

Seu desafio será desenvolver um programa com arquivos .csv para realizar o cadastro, leitura e consulta de notas dos estudantes.

O programa deve apresentar o menu:

- **[1] – Cadastrar aluno e nota:** Permitir inserir nome do aluno, idade e nota.
- **[2] – Lista alunos:** Exibe todos os alunos cadastrados até o momento no arquivo csv.
- **[3] – Lista alunos com nota acima de 8:** Exibe todos os alunos cadastrados no arquivo csv com a nota superior a 8.
- **[0] – Sair:** Encerrar o programa.

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .json



Manipulação de Arquivos JSON no Python

A biblioteca json do **Python** permite trabalhar com arquivos .json, muito utilizados para **armazenar e trocar dados entre sistemas, APIs e aplicações web**.

Os dados em **JSON** são organizados em **formato de chave e valor**, semelhantes aos **dicionários** do Python.

Aplicações do JSON:

-  APIs e aplicações web
-  Aplicativos mobile
-  Arquivos de configuração
-  Troca de dados entre sistemas

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .json



Leitura de Arquivos JSON

A **leitura** é feita utilizando `json.load()`, permitindo transformar o conteúdo do arquivo em estruturas do **Python**.

Função	Descrição
<code>json.load()</code>	Lê dados de um arquivo JSON
<code>json.loads()</code>	Lê dados JSON em formato de texto/string



Exemplo:

```
1 import json
2
3 with open("dados.json", "r") as arquivo:
4     dados = json.load(arquivo)
5
6 print(dados)
7 print(dados["nome"])
```

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .json



Exemplo — `json.loads()`

A função `json.loads()` converte um texto **JSON** em uma estrutura do **Python**, como um dicionário.

Exemplo:

```
1 import json
2
3 texto_json = '{"nome": "Carlos", "idade": 17, "nota": 8.5}'
4
5 dados = json.loads(texto_json)
6
7 print(dados)
8 print(dados["nome"])
```

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .json



Escrita de Arquivos JSON

A escrita é feita utilizando `json.dump()`, permitindo salvar estruturas do Python em formato JSON.

Função	Descrição
<code>json.dump()</code>	Salva dados em um arquivo JSON
<code>json.dumps()</code>	Converte dados para texto JSON



Exemplo:

```
1 import json
2
3 dados = {
4     "nome": "Ana",
5     "idade": 16,
6     "nota": 9.5
7 }
8
9 with open("dados.json", "w") as arquivo:
10     json.dump(dados, arquivo, indent=4)
```

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .json



Exemplo — `json.dumps()`

A função `json.dumps()` converte um dicionário Python em texto **JSON**.



Exemplo:

```
1 import json
2
3 aluno = {
4     "nome": "Ana",
5     "idade": 16,
6     "nota": 9.0
7 }
8
9 texto_json = json.dumps(aluno, indent=4)
10
11 print(texto_json)
```

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .json

Vamos
realizar
juntos!



Exemplo Prático — Cadastro de Alunos com JSON

Parte I — Salvando Dados no JSON

O programa abaixo cria uma lista de alunos e salva as informações no arquivo alunos.json.

```
03-Json.py X
material > 03-Json.py > ...
1  import json
2
3  # Lista de alunos
4  alunos = [
5      {
6          "nome": "Ana",
7          "idade": 16,
8          "nota": 9.0
9      },
10     {
11         "nome": "Carlos",
12         "idade": 17,
13         "nota": 7.5
14     },
15     {
16         "nome": "Marina",
17         "idade": 15,
18         "nota": 8.8
19     }
20 ]
21
```

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .json

Vamos
realizar
juntos!



Exemplo Prático — Cadastro de Alunos com JSON



Parte I — Salvando Dados no JSON

O programa abaixo cria uma lista de alunos e salva as informações no arquivo alunos.json.

```
22 # Salvando no arquivo JSON
23 with open("alunos.json", "w") as arquivo:
24     json.dump(alunos, arquivo, indent=4)
25
26 print("Dados salvos com sucesso!")
```

PROGRAMAÇÃO EM PYTHON



Automação com Arquivos .json

Vamos
realizar
juntos!



Exemplo Prático — Cadastro de Alunos com JSON

 Parte 2 — Lendo os Dados do JSON

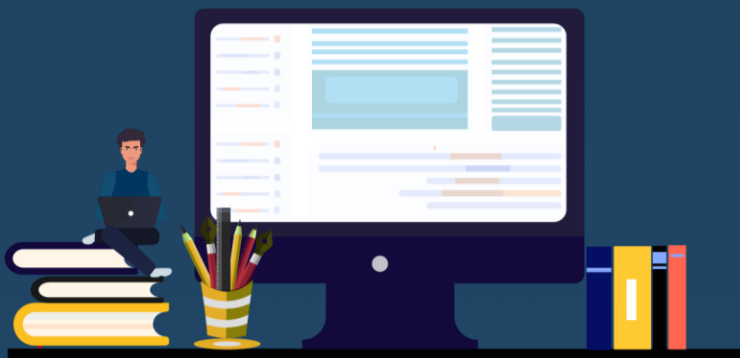
Agora vamos abrir o arquivo e mostrar as informações no terminal.

```
04-Json-leitura.py ×
material > 04-Json-leitura.py > ...
1  import json
2
3  # Abrindo o arquivo JSON
4  with open("alunos.json", "r") as arquivo:
5      alunos = json.load(arquivo)
6
7  # Exibindo os dados
8  for aluno in alunos:
9      print("Nome:", aluno["nome"])
10     print("Idade:", aluno["idade"])
11     print("Nota:", aluno["nota"])
12     print("-" * 20)
13
14
```

PROGRAMAÇÃO EM PYTHON



Apresentação da situação de aprendizagem 2



Sistema de Produtos - **FABMARKET**

Imagine que você foi contratado para desenvolver um pequeno sistema de cadastro de produtos para um supermercado chamado **FABMARKET**. Atualmente, o controle dos produtos é feito manualmente, dificultando a organização das informações, a consulta de preços e o gerenciamento do estoque.

Seu desafio será desenvolver um sistema utilizando o Python e arquivos no formato .json para armazenar e manipular os dados dos produtos de forma automatizada.

O programa deverá permitir as seguintes operações:

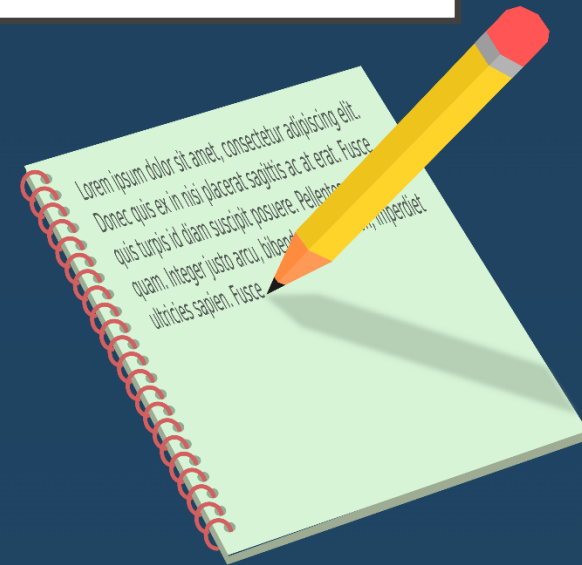
- **Cadastro de produtos:** O usuário deverá conseguir cadastrar novos produtos informando: nome do produto, categoria, quantidade em estoque e preço.
- **Listagem de produtos:** O sistema deverá exibir todos os produtos cadastrados no arquivo JSON.
- **Consulta de preços:** O programa deverá permitir visualizar apenas os produtos com valor acima de um determinado preço definido pelo usuário.
- **Encerramento do sistema:** O usuário poderá finalizar o programa.

EXERCÍCIOS

Desenvolva um programa que solicite ao usuário os dados de um jogo (nome, gênero, nota, plataforma) e armazene-os em um arquivo JSON chamado “jogos.json”.

Desafios Extras (Opcional):

- Criar um menu de opções
- Busca de jogos por nome
- Remover jogos do JSON
- Atualizar notas
- Melhorar o visual do terminal



REFERÊNCIAS

Python Fluente. *Python Fluente: Programação Clara, Concisa e Eficaz*. 2. ed. Porto Alegre: Bookman, 2023.

Introdução à Programação com Python. *Introdução à Programação com Python: Algoritmos e Lógica de Programação para Iniciantes*. 3. ed. São Paulo: Novatec, 2019.

Use a Cabeça! Python. *Use a Cabeça! Python*. Rio de Janeiro: Alta Books, 2018.

Python Software Foundation. *Documentação Oficial do Python*. Disponível em: python.org. Acesso em: 15 maio 2026.

JSON. *Introducing JSON*. Disponível em: [JSON.org](https://json.org). Acesso em: 15 maio 2026.

Importante:

Os conteúdos disponibilizados são específicos para este curso/turma, a divulgação ou reprodução do material para outras pessoas/organização não é autorizada.



wilck.gomes@sp.senai.br



Wilck Gomes de Oliveira



Instrutor de Formação Profissional III

